

Peretto

Marcos  
Luciano

PDI — ATIVIDADE 02 · REGISTRO DE ENTREGA

# APIs Modulares de Missao Critica (**FastAPI**) com Paginacao, Cache e Rate Limiting

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company — Automação & Infraestrutura

Módulo: 03 · Fullstack e Modelagem de Dados

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

`cursor pagination / Redis / rate limit`

## 1. Contexto

A API de pedidos da FV cresceu sem padrão: o endpoint /pedidos devolvia a lista inteira com LIMIT/OFFSET, o banco PostgreSQL sofria em páginas profundas e o mesmo JSON era recalculado dezenas de vezes por minuto. Quando o tráfego subiu, o endpoint virou gargalo e começou a derrubar o Postgres.

### ANALOGIA DO CARDÁPIO GIGANTE

Imagine um restaurante que, para te mostrar a página 50 do cardápio, relê TODAS as 5.000 receitas desde a primeira. O garçom (banco) cansa. A paginação por cursor é pedir: 'me traga

o que vem DEPOIS do prato X' — o garçom vai direto lá, sem reler tudo. E se 100 clientes pedem o mesmo prato popular? O cache é a foto do prato no balcão: ninguém precisa ir à cozinha de novo.

## 2. Diagnóstico

Com OFFSET grande, o Postgres varre e descarta milhares de linhas antes de devolver 20. Sem cache, consultas idênticas martelam o banco. Sem limite de taxa, um único cliente (ou bot) derruba a operação para todos.



**Raiz do problema:** paginação deslocada (offset) + ausência de cache de leitura + nenhum limite de taxa. O custo de CPU do banco escala com o número de páginas e com a repetição de consultas idênticas.

## 3. Solução

Reescrever o endpoint com **cursor pagination** (ordenado por (created\_at, id)), colocar um **cache Redis** na resposta (TTL curto, invalidado por escrita) e proteger a rota com **rate limiting** por cliente via token no Redis. O Postgres só é tocado quando o cache erra.

## 4. Como funciona (pipeline)

Fluxo de leitura paginada com cache e rate limit

FLUXO EM EXECUÇÃO

1

**Validação de token + rate limit**

Middleware checa o bucket do cliente no Redis; se estourou o limite, responde 429.

ok

2

**Cálculo da chave de cache**

Monta a chave pedidos:{cliente}:{cursor}:{size} a partir dos parâmetros.

warn

3

**Leitura do Redis**

Se a chave existe (hit), devolve o JSON em ~1ms sem tocar o banco.

ok

4

**Query cursor no Postgres**

No miss, executa `WHERE (created_at,id) > (?,?) ORDER BY created_at,id LIMIT ?`.

audit

5

**Grava cache + responde**

Salva a resposta no Redis com EX 30 e devolve com o próximo cursor.

ok

Escrita em /pedidos faz `DEL pedidos:{cliente}:*` para invalidar o cache daquele cliente (evita resposta stale).

## 5. Antes vs Depois

CENÁRIO

ANTES (OFFSET)

DEPOIS (CURSOR+CACHE)

|                   |                      |                       |
|-------------------|----------------------|-----------------------|
| Página 1000       | Varre 20k linhas     | Seek direto no índice |
| Leitura repetida  | Bate no banco sempre | Hit em Redis (~1ms)   |
| Abuso de API      | Sem proteção         | 429 após limite       |
| Carga no Postgres | Alta e desnecessária | Só em cache miss      |

## 6. Entregas

- **Endpoint** GET /pedidos com cursor pagination e schema Pydantic de resposta.
- **Middleware** rate\_limit.py: token bucket no Redis por cliente.
- **Cache layer** cache.py: get/set com invalidação por escrita.
- **Migration** 002\_cursor\_idx.sql: índice composto (created\_at, id).
- **Testes** test\_pagination.py: cobre primeira/última página e cache hit/miss.

## 7. Métricas



**Meta:** p95 de /pedidos abaixo de 120ms e >90% de cache hit, com zero incidente de sobrecarga no Postgres.

## 8. Status final

**NÃO PUBLICADO** — Desenvolvido e em homologação. Aguarda revisão antes de produção.